# Azure DevOps and Azure Pipelines

## A practical beginner's guide for your ASP.NET Core application

Learn to read, write, run, and debug your CI/CD pipeline using the **cicd** project as the working example.

| Your application | Your delivery workflow |
| --- | --- |
| ASP.NET Core 10 MVC | Build and test the solution |
| EF Core + SQL Server / Azure SQL | Package a single deployable ZIP |
| Identity users, roles, permission claims | Deploy the ZIP to staging |
| Azure App Service on Windows | Verify readiness, then swap into production |

**Reading order:** start with the vocabulary and execution flow, read the annotated YAML, then follow the configuration and troubleshooting chapters.

**Important:** this is a learning document, not a configuration change. The pipeline snapshot has master push triggers enabled. Your current slots share one SQL database. Database migrations and seeding are separate operations.

Prepared for your project on 13 November 2026. All connection strings and passwords in examples are placeholders. Never insert real secrets into committed YAML.

# 01 / The systems and vocabulary

Azure DevOps organizes your software delivery work. Microsoft Azure hosts the running application and database. They are separate services connected by an automation identity.

| Term | Meaning in your project |
| --- | --- |
| Repository | Git stores source files, commits, branches, and the YAML. Azure Repos or GitHub can host it. |
| Azure Pipelines | The service that reads YAML and schedules build, test, and deployment work. |
| CI | Continuous integration: automatically restore, compile, and test changes. |
| Continuous delivery | Prepare a tested release; a person may approve production deployment. |
| Continuous deployment | Automatically release a successful candidate to production. |
| App Service | The Azure service hosting your ASP.NET Core app. |
| Deployment slot | A separately hosted version of the app, such as staging, under the parent Web App. |
| Azure SQL Database | Stores Identity users, password hashes, roles, claims, and application data. |

## Four identities that beginners often confuse

Your Microsoft account signs you into the portals. A service connection lets the pipeline operate Azure resources. The SQL login lets the app connect to its database. Identity Admin/Manager/Customer accounts let people sign into your app. These are independent permissions.

For example, a package can deploy successfully using the service connection while the application fails with SQL Login failed for user. Successful deployment does not prove database authentication works.

# 02 / How a pipeline is organized

```
Pipeline
  Stage
    Job or deployment job
      Steps
        Task or script
```

| Concept | Simple explanation |
| --- | --- |
| Stage | A major phase. Yours are BuildTest, Staging, and Production. |
| Job | A collection of steps executed by an agent. Build is a normal job. |
| Deployment job | A deployment-aware job linked to an Azure DevOps environment. |
| Step | One ordered operation: restore, build, test, or deploy. |
| Task | Reusable automation, such as DotNetCoreCLI@2. |
| Script | Custom commands, such as the PowerShell readiness loop. |
| Agent | The machine executing a job. It is not the Azure Web App. |
| Pool | The collection/source of agents available to execute jobs. |
| Artifact | A saved output from a run; your artifact contains webapp.zip. |
| Environment | Azure DevOps deployment history and controls, such as approvals. |

Steps normally execute in sequence inside a job. Jobs can run in parallel unless dependencies restrict them. Your stages explicitly depend on each other, establishing build, staging, then production.

Microsoft-hosted jobs use fresh machines. A later job cannot rely on the build job's local files. Upload and download artifacts to move outputs between jobs.

# 03 / Reading YAML and expressions

| Syntax | Meaning |
| --- | --- |
| key: value | Assign a property value. Example: command: build. |
| Indentation | Nest one configuration inside another. Use spaces, not tabs. |
| - | Start a list item: one stage, job, or task. |
| # | A comment ignored by the pipeline. |
| true / false | Boolean values: enabled / disabled. |
| \| | Start a multiline text block, such as an embedded script. |
| $(webAppName) | Pipeline macro: replaced by the variable value before the task executes. |
| @2 | Major task version. DotNetCoreCLI@2 does not mean .NET version 2. |
| displayName | Friendly UI label. Internal stage/job identifiers are used by dependencies. |

## Three expression styles in your health-check step

```
$(webAppName)          # Azure Pipelines substitutes a value
$hostname             # PowerShell variable
$($hostname.Trim())    # PowerShell expression inside a string
```

## Branch conditions

```
condition: and(succeeded(), eq(variables['Build.SourceBranch'], 'refs/heads/master'))
```

**and** requires both checks to be true. **succeeded()** checks successful dependencies. **eq** compares values. **Build.SourceBranch** is the full Git reference. A manual feature-branch run can build but skip deployment. If you rename master to main, update both triggers and these conditions.

Other expression forms exist: ${{ ... }} is generally used for template/compile-time expansion, while $[ ... ] is used for runtime expressions. You do not need them for this pipeline. Learn them when introducing templates and computed variables.

# 04 / The execution flow

### 1. Push or manual run

A push matching the active master trigger queues a run. Run pipeline also allows a manual run.

### 2. Evaluate YAML

Azure DevOps determines stages, dependencies, variables, required resources, and conditions.

### 3. Assign build agent

A hosted Windows agent checks out the selected source commit and selects a .NET 10 SDK.

### 4. Restore and build

NuGet dependencies resolve; application and tests compile in Release configuration.

### 5. Test

The test project executes; failed tests stop the normal success path.

### 6. Publish and archive

Deployable web files are produced, then compressed into webapp.zip.

### 7. Upload artifact

The ZIP is stored as the run artifact named drop.

### 8. Deploy staging

A deployment job downloads that same run's package and deploys it to the staging slot.

### 9. Check readiness

The agent requests /health/ready. Your app queries Identity tables and checks roles.

### 10. Approve if configured

The production environment can pause the next stage for approval.

### 11. Promote

A slot swap moves the candidate app into production; there is no second build.

### 12. Record results

The run retains logs, test reports, artifact references, and deployment history.

# 05 / Annotated pipeline: every configuration line

The following is the complete current YAML with explanatory comments. Long lines are visually wrapped to fit the page. Refer to your repository for copyable commands. A commented-out setting has no effect.

```yaml
# Temporarily disable push-triggered runs; manual runs remain available.

#trigger: none

# Restore this configuration to enable automatic runs on master:

# Enable automatic CI runs for matching pushes.
trigger:

  # Batch newer pushes while a CI run is active; not a universal deployment lock.
  batch: true

  # Define branch filters for automatic runs.
  branches:

    # List the branches that should trigger CI.
    include:

    # Only pushes to master match this filter.
    - master

# Choose agent infrastructure for jobs that do not override this setting.
pool:

  # Use a Microsoft-hosted Windows image; the image can change over time.
  vmImage: 'windows-latest'

# Define reusable pipeline values, not application runtime settings.
variables:

  # Compile the Release configuration; this is not ASPNETCORE_ENVIRONMENT.
  buildConfiguration: 'Release'

  # Name of the authorized Azure DevOps service connection.
  azureServiceConnection: 'azure-cicd-connection'

  # Parent App Service resource name, not its URL.
  webAppName: 'cicdwebapp'

  # Resource group containing the Web App.
  resourceGroupName: 'cicdrg'

  # Windows App Service; use webAppLinux for Linux.
  webAppType: 'webApp' # Use webAppLinux for a Linux App Service.

  # Existing slot under the parent Web App.
  stagingSlot: 'staging'

# List the major pipeline phases.
stages:

# Internal build/test stage identifier.
- stage: BuildTest

  # Friendly label displayed in the pipeline run UI.
  displayName: 'Build and test'

  # Jobs belonging to this stage.
  jobs:

  # Normal agent job for compilation and packaging.
  - job: Build

    # Ordered tasks or scripts in this job/lifecycle hook.
    steps:
```

```
    # Select a compatible .NET SDK using major task version 2.
    - task: UseDotNet@2

      # Task-specific configuration follows; input names depend on the task.
      inputs:

        # SDK contains compiler/build tools; a runtime alone cannot build.
        packageType: 'sdk'

        # Use a .NET 10 SDK version; x allows version resolution.
        version: '10.0.x'

    # Run a dotnet CLI operation using task major version 2.
    - task: DotNetCoreCLI@2

      # Friendly label displayed in the pipeline run UI.
      displayName: 'Restore solution'

      # Task-specific configuration follows; input names depend on the task.
      inputs:

        # Resolve NuGet dependencies; does not migrate SQL.
        command: 'restore'

        # Operate on the checked-in solution and its referenced projects.
        projects: 'cicd.sln'

    # Run a dotnet CLI operation using task major version 2.
    - task: DotNetCoreCLI@2

      # Friendly label displayed in the pipeline run UI.
      displayName: 'Build solution'

      # Task-specific configuration follows; input names depend on the task.
      inputs:

        # Compile projects into assemblies.
        command: 'build'

        # Operate on the checked-in solution and its referenced projects.
        projects: 'cicd.sln'

        # Use Release; reuse dependency restore from the previous step.
        arguments: '--configuration $(buildConfiguration) --no-restore'

    # Run a dotnet CLI operation using task major version 2.
    - task: DotNetCoreCLI@2

      # Friendly label displayed in the pipeline run UI.
      displayName: 'Run unit tests'

      # Task-specific configuration follows; input names depend on the task.
      inputs:

        # Execute the configured test project.
        command: 'test'

        # Run this test project; it must be inside the repository.
        projects: 'tests/cicdtest/cicdtest.csproj'

        # Use the Release assemblies already built in this job.
        arguments: '--configuration $(buildConfiguration) --no-build'

        # Upload test results to the run's Tests view.
        publishTestResults: true

    # Run a dotnet CLI operation using task major version 2.
    - task: DotNetCoreCLI@2

      # Friendly label displayed in the pipeline run UI.
      displayName: 'Package web app'

      # Task-specific configuration follows; input names depend on the task.
      inputs:
```

```
          # Produce deployable files; does not itself deploy to Azure.
          command: 'publish'

          # Use the explicit projects input instead of automatic web-project discovery.
          publishWebProjects: false

          # Publish only the web application project.
          projects: 'cicd.csproj'

          # Publish existing Release build into the known artifact-preparation directory.
          arguments: '--configuration $(buildConfiguration) --no-build --output
              $(Build.ArtifactStagingDirectory)/webapp'

          # Do not let this task create a ZIP; ArchiveFiles does it explicitly.
          zipAfterPublish: false

          # Keep the specified publish output path without project-name modification.
          modifyOutputPath: false

      # Create an archive from the published app directory.
      - task: ArchiveFiles@2

        # Friendly label displayed in the pipeline run UI.
        displayName: 'Create webapp.zip'

        # Task-specific configuration follows; input names depend on the task.
        inputs:

          # Source directory containing deployable files.
          rootFolderOrFile: '$(Build.ArtifactStagingDirectory)/webapp'

          # Place DLL/config/static files directly at the ZIP root.
          includeRootFolder: false

          # Use ZIP format for App Service deployment.
          archiveType: 'zip'

          # Exact output ZIP path used by the next task.
          archiveFile: '$(Build.ArtifactStagingDirectory)/webapp.zip'

          # Replace any existing archive at that path.
          replaceExistingArchive: true

      # Upload the ZIP to Azure Pipelines storage for this run.
      - task: PublishPipelineArtifact@1

        # Task-specific configuration follows; input names depend on the task.
        inputs:

          # File to upload; it must already exist.
          targetPath: '$(Build.ArtifactStagingDirectory)/webapp.zip'

          # Artifact label; drop is a chosen name, not special syntax.
          artifact: 'drop'

  # Stage for staging deployment and verification.
  - stage: Staging

    # Friendly label displayed in the pipeline run UI.
    displayName: 'Deploy to staging slot'

    # Wait for the build/test stage before evaluating staging work.
    dependsOn: BuildTest

    # Run only when dependencies succeed AND the branch is refs/heads/master.
    condition: and(succeeded(), eq(variables['Build.SourceBranch'], 'refs/heads/master'))

    # Jobs belonging to this stage.
    jobs:

    # Deployment job with environment history and checks.
    - deployment: DeployStaging
```

```
# Azure DevOps environment; does not create/select an Azure slot.
environment: 'cicd-staging'

# Choose deployment execution behavior.
strategy:

  # Execute the deployment lifecycle once.
  runOnce:

    # Deployment lifecycle hook containing the steps.
    deploy:

      # Ordered tasks or scripts in this job/lifecycle hook.
      steps:

      # Download artifacts from this exact pipeline run.
      - download: current

        # Select the artifact named drop.
        artifact: drop

      # Deploy application content to an existing App Service.
      - task: AzureWebApp@1

        # Friendly label displayed in the pipeline run UI.
        displayName: 'Deploy package to staging'

        # Task-specific configuration follows; input names depend on the task.
        inputs:

          # Service connection name, despite the input being called azureSubscription.
          azureSubscription: '$(azureServiceConnection)'

          # Choose Windows or Linux deployment behavior.
          appType: '$(webAppType)'

          # Existing parent Web App resource.
          appName: '$(webAppName)'

          # Deploy to a named slot rather than the default production target.
          deployToSlotOrASE: true

          # Resource group used to locate the slot.
          resourceGroupName: '$(resourceGroupName)'

          # Named staging slot to receive the app.
          slotName: '$(stagingSlot)'

          # Downloaded package from this run's artifact.
          package: '$(Pipeline.Workspace)/drop/webapp.zip'

          # Use App Service ZIP deployment; no automatic EF migration.
          deploymentMethod: 'zipDeploy'

      # Authenticate Azure CLI through the service connection and run a script.
      - task: AzureCLI@2

        # Friendly label displayed in the pipeline run UI.
        displayName: 'Check staging responds successfully'

        # Task-specific configuration follows; input names depend on the task.
        inputs:

          # Service connection name, despite the input being called azureSubscription.
          azureSubscription: '$(azureServiceConnection)'

          # Use PowerShell Core (supports SkipHttpErrorCheck).
          scriptType: 'pscore'

          # The script is embedded below, rather than in a separate file.
          scriptLocation: 'inlineScript'

          # Preserve the following indented lines as a multiline script.
          inlineScript: |
```

```
                    # Query the actual staging hostname; --query selects a field, --output tsv
                        returns plain text.
                    $hostname = az webapp show --resource-group "$(resourceGroupName)" --name
                        "$(webAppName)" --slot "$(stagingSlot)" --query defaultHostName --output tsv

                    # Reject failed Azure CLI execution or an empty hostname.
                    if ($LASTEXITCODE -ne 0 -or [string]::IsNullOrWhiteSpace($hostname)) { throw
                        'Cannot resolve staging hostname.' }

                    # Trim whitespace and construct the readiness URL.
                    $url = "https://$($hostname.Trim())/health/ready"

                    # Start at 1; repeat while attempt <= 12; increment each time.
                    for ($attempt = 1; $attempt -le 12; $attempt++) {

                      # Attempt the HTTP request; catch handles thrown request errors.
                      try {

                        # HTTP request; timeout is 30 seconds; inspect non-2xx responses without
                            throwing.
                        $response = Invoke-WebRequest -Uri $url -TimeoutSec 30 -SkipHttpErrorCheck

                        # HTTP 200 passes the check; exit 0 reports success.
                        if ($response.StatusCode -eq 200) { Write-Host 'Staging check passed.'; exit
                             0 }

                        # Print status and attempt number; [int] converts the status to a number.
                        Write-Host "Staging returned HTTP $([int]$response.StatusCode) (attempt
                            $attempt)."

                      # Report transport exceptions, such as DNS, TLS, or timeout failures.
                      } catch { Write-Host "Staging request failed before a response was received
                          (attempt $attempt). Check slot availability, DNS, TLS, and network
                          access." }

                      # Wait ten seconds before retrying; request time is additional.
                      Start-Sleep -Seconds 10

                    # Close the preceding PowerShell block.
                    }

                    # All attempts failed: fail this task and prevent production promotion.
                    throw 'Staging readiness failed; production promotion stopped. Check staging Log
                         stream, ConnectionStrings__DefaultConnection, SQL access, and database
                        initialization. See README.md.'

  # Stage that promotes the staged app.
  - stage: Production

    # Friendly label displayed in the pipeline run UI.
    displayName: 'Promote staging to production'

    # Wait for staging and its readiness check.
    dependsOn: Staging

    # Run only when dependencies succeed AND the branch is refs/heads/master.
    condition: and(succeeded(), eq(variables['Build.SourceBranch'], 'refs/heads/master'))

    # Jobs belonging to this stage.
    jobs:

    # Deployment job performing production promotion.
    - deployment: PromoteProduction

      # Optional approval: configure Approvals and checks on this environment in Azure DevOps.

      # Azure DevOps environment on which to configure production approval.
      environment: 'cicd-production'

      # Choose deployment execution behavior.
      strategy:

        # Execute the deployment lifecycle once.
        runOnce:
```

```
# Deployment lifecycle hook containing the steps.
deploy:

  # Ordered tasks or scripts in this job/lifecycle hook.
  steps:

  # Disable automatic artifact download; a slot swap needs no ZIP.
  - download: none

  # Manage an existing App Service; @0 is the task major version.
  - task: AzureAppServiceManage@0

    # Friendly label displayed in the pipeline run UI.
    displayName: 'Swap staging into production'

    # Task-specific configuration follows; input names depend on the task.
    inputs:

      # Service connection name, despite the input being called azureSubscription.
      azureSubscription: '$(azureServiceConnection)'

      # Promote by swapping slots rather than building another package.
      Action: 'Swap Slots'

      # Parent Web App whose slots are swapped.
      WebAppName: '$(webAppName)'

      # Resource group containing the Web App.
      ResourceGroupName: '$(resourceGroupName)'

      # Candidate source slot containing the tested app.
      SourceSlot: '$(stagingSlot)'

      # Use production as the swap destination.
      SwapWithProduction: true
```

# 06 / Commands, directories, and package structure

## What the .NET tasks actually do

```
dotnet restore cicd.sln
dotnet build cicd.sln --configuration Release --no-restore
dotnet test tests/cicdtest/cicdtest.csproj --configuration Release --no-build
dotnet publish cicd.csproj --configuration Release --no-build --output <folder>
```

Restore resolves package references such as EF Core, Identity, and xUnit. Build compiles the solution. Test executes the test assembly. Publish prepares files for hosting. Only the later AzureWebApp task deploys those files to Azure.

--no-restore avoids repeating an earlier restore. --no-build reuses the assemblies compiled in this same job. The build configuration must match between these steps. The test task also handles result publication when enabled.

| Variable | Purpose |
| --- | --- |
| Build.ArtifactStagingDirectory | Temporary build-job directory for files being prepared for upload. Not your Azure staging slot. |
| Pipeline.Workspace | Current job workspace. Downloaded drop artifact lives below this location. |
| Build.SourceBranch | Full Git reference for the selected commit, such as refs/heads/master. |

## Required ZIP layout

```
webapp.zip
  cicd.dll
  web.config
  appsettings.json
  wwwroot/
```

includeRootFolder: false avoids an extra webapp/ directory above these files. The upload and deployment paths must agree on the filename webapp.zip. Build-agent files disappear with the agent, so saving an artifact is essential.

## Different meanings of publish

dotnet publish prepares files. Publish test results uploads reports. Publish pipeline artifact uploads a package. Visual Studio Publish is a separate interactive workflow that can build and deploy. These operations are related but not interchangeable.

# 07 / Azure DevOps portal setup

## Connect the repository and create the pipeline

Open your Azure DevOps project > Pipelines > Pipelines > New pipeline. Select Azure Repos Git or the actual repository provider. Select the repository, choose Existing Azure Pipelines YAML file, and select azure-pipelines.yml on the intended branch. Review before running. For later runs, use Run pipeline and choose a branch.

## Create the service connection

Project settings > Service connections > New service connection > Azure Resource Manager. Choose App registration (automatic) with Workload identity federation when your permissions allow it. Select the subscription and appropriate resource scope. Name it azure-cicd-connection. Save and authorize the intended pipeline. Limit scope and pipeline access to what the deployment needs.

Workload identity federation avoids storing a long-lived client secret. The service connection allows Azure management operations; it does not automatically grant SQL database access to your application.

## Create deployment environments

Pipelines > Environments > New environment. Create cicd-staging and cicd-production. Choose None for resources when using the environment to organize this App Service deployment. Environment names must match the YAML.

## Require production approval

Pipelines > Environments > cicd-production > menu > Approvals and checks > add Approvals. Choose approvers, configure timeout/self-approval preferences, and save. Azure DevOps waits for required checks before allowing the stage using the environment to run. The YAML comment about approval does not create the check.

## Inspect a run

Pipelines > your pipeline > select a run. Open the stage, job, then the first failing task. Inspect Tests for published results and the run artifact list for drop. Check the source branch and commit before retrying a release.

## Protect the release branch

For Azure Repos: Repos > Branches > master > Branch policies > Build validation. Configure pull-request validation before merging. PR validation checks proposed code; the push trigger runs after a matching push. Provider-specific PR behavior differs for GitHub.

# 08 / Variables, secrets, and runtime configuration

## Inline values versus variable groups

Inline variables are easy to review with source code. A variable group centralizes shared configuration. Create one under Pipelines > Library > + Variable group. Add values, save, and authorize your pipeline through Pipeline permissions. Use the lock control for secret values.

```
variables:
- group: cicd-settings
- name: buildConfiguration
  value: 'Release'
```

This list syntax replaces the original mapping-style variables block. Do not combine the two YAML forms incorrectly. Variable groups can also retrieve supported secrets from Azure Key Vault.

## Explicitly map a secret into a script

```
env:
  ConnectionStrings__DefaultConnection: $(stagingSqlConnection)
```

This illustration passes a pipeline secret into a process environment. It does not configure your Web App. Never echo secrets. Secret masking helps, but it is not a guarantee against every form of accidental disclosure.

| Location | Who uses the value? |
| --- | --- |
| YAML / variable group | Authorized pipeline tasks. |
| App Service application settings | The deployed web application. |
| Local user secrets | Local application configuration when the relevant environment loads it. |
| ASPNETCORE_ENVIRONMENT | Runtime environment selection; independent of Release compilation. |
| Azure DevOps environment | Deployment history and release controls, not app runtime configuration. |

App Service uses ConnectionStrings__DefaultConnection for the configuration path ConnectionStrings:DefaultConnection. Your code reads this with GetConnectionString("DefaultConnection"). The double underscore is significant.

# 09 / App Service slots and swapping

## Configure Azure resources

Azure Portal > App Services > cicdwebapp. Verify the .NET runtime, then Deployment slots > Add slot to create staging if needed. Deployment slots require a supporting App Service tier. Keep auto swap disabled because the pipeline performs promotion after its own checks.

In each slot: Environment variables > App settings > add/edit ConnectionStrings__DefaultConnection. Use the correct database connection. Enable Deployment slot setting and apply/save. A deployment slot setting stays associated with the slot during a swap.

## What promotion does

| Before swap | After swap |
| --- | --- |
| Staging contains new candidate app | Staging contains previous production app |
| Production contains previous app | Production serves the new candidate app |

The production job does not rebuild or deploy a second ZIP. It promotes the staged app. Azure prepares/warm-ups the source as part of the swap and applies documented configuration rules. Keep production as the destination.

## Your current shared database

Both slots use the same SQL database, so they share users, roles, claims, and application data. Use the same connection string in both slots if that is intentional. Keeping Deployment slot setting checked is still appropriate and supports separation later.

Testing staging can modify production data. A migration changes the shared schema immediately. Swapping application slots does not swap database content or undo a migration. A separate staging database is the stronger production approach.

## Release overlap

batch: true reduces overlapping push-triggered CI runs. It does not stop manually queued releases or other pipelines from using staging. Use an appropriate release-wide concurrency strategy for production. A lock scoped only to the final swap cannot protect a candidate that another run already overwrote in staging.

# 10 / SQL, Entity Framework, and Identity

| Operation | When it happens |
| --- | --- |
| Restore | Resolve EF and Identity dependencies during the build. |
| Migration | Create/update SQL schema through the explicit initializer or a controlled migration step. |
| Seeding | Create missing roles, claims, users, and memberships during initialization. |
| Runtime queries | The Web App authenticates users and reads/writes application data. |

## Your explicit initialization command

```
dotnet run --configuration Release --no-launch-profile -- --initialize-database
```

The final argument is handled by Program.cs. It runs migrations, runs IdentitySeeder, and exits. Your local Initialize-Staging.ps1 script supplies temporary configuration and prompts for credentials. The current pipeline does not call either initializer.

Identity UserManager hashes a new user password before writing AspNetUsers.PasswordHash. Roles and permission claims are stored in Identity tables. On sign-in, role claims become part of the authenticated principal. Existing user passwords are preserved by your repeatable seeder.

## Recommended database release approach

Create a reviewed migration script or bundle and execute it in a controlled step with schema-change permission. Avoid having every web-server instance apply schema changes on startup. Use a more restricted database identity for ordinary runtime operations where practical.

For rolling/slot releases, changes must support both old and new code. Add a nullable column first, deploy compatible code, and remove old structures only after no deployed version needs them. An immediate column rename/drop can break production while staging is still being validated.

## What your tests prove

Your Identity unit tests use isolated EF in-memory storage. They check seeding, role memberships, permissions, and repository behavior. They do not prove Azure SQL credentials, SQL translation, firewall rules, or migration correctness. Add integration tests against a separate SQL test database to verify those boundaries.

# 11 / Readiness checks and failure diagnosis

## How the readiness script works

AzureCLI authenticates using the service connection. az webapp show obtains the actual staging hostname. --query defaultHostName selects one field; --output tsv returns plain text. This avoids guessing Azure-generated hostnames.

$LASTEXITCODE checks the native Azure CLI exit code. IsNullOrWhiteSpace rejects missing output. Trim removes surrounding whitespace. The script requests /health/ready up to 12 times, allowing 30 seconds per request and waiting 10 seconds between attempts. Total duration can exceed two minutes.

HTTP 200 exits successfully. Non-200 responses are logged by number. Transport exceptions are caught separately. After all attempts fail, throw fails the task and blocks production. The request comes from the pipeline agent, so private endpoints need compatible network access.

## Interpret the status, then find the exception

| Observation | Next action |
| --- | --- |
| HTTP 503 on /health/ready | Read application logs. The Identity database may be unreachable or lack schema/roles. |
| SQL Login failed for user | Check the same server, database, username, password, and database access in SSMS. Fix the slot connection setting. |
| Invalid object AspNetUsers | Apply migrations to the database actually used by the slot. |
| HTTP 500 on login | Inspect the .NET exception; a generic IIS HTML error is not the root cause. |
| HTTP 403 / network exception | Check endpoint restrictions, authentication, DNS/TLS, and agent network access. |

## Get useful application logs

Azure Portal > staging slot > App Service logs > Application logging (Filesystem) On > Error > Save. Open Log stream and reproduce the failure. Your application explicitly registers AddAzureWebAppDiagnostics. Keep the deployed app in Production mode; inspect server logs rather than exposing detailed exceptions to users.

# 12 / Common beginner mistakes

| Problem | Explanation / resolution |
| --- | --- |
| YAML parse failure | Check spaces, indentation, colons, and list markers. |
| No push-triggered run | Check trigger: none, branch filters, selected YAML, and pipeline UI overrides. |
| Deployment skipped | The branch condition may not match refs/heads/master. |
| No agent starts | Check pool access and organization parallel-job capacity. |
| SDK mismatch | Match UseDotNet with net10.0; SDK and hosted runtime are separate. |
| Restore failure | Check package feeds, authentication, NuGet.Config, package versions, and network. |
| Project not found | Commit referenced projects; avoid paths pointing outside the repository. |
| Artifact path missing | Publish, archive, upload, and download must agree on webapp.zip. |
| Service connection denied | Check exact name, pipeline authorization, subscription/scope, and Azure role permissions. |
| Slot not found | Create staging under the correct Web App and verify resource group. |
| Approval wait | Inspect the production environment's pending approvals/checks and authorized approvers. |
| Database changes missing | Deploying code does not apply EF migrations or run seeding. |

Start with the first failing task, not the last red summary. Script failed with exit code 1 only reports failure. The earlier exception, such as Login failed for user, identifies what to fix. Retrying without changing the cause usually produces the same failure.

# 13 / Production habits and practice plan

## Adopt these habits progressively

Protect master with pull requests and validation. Build once and promote the tested package. Require production approval where appropriate. Use workload identity federation and limited Azure permissions. Keep secrets out of Git. Separate staging and production data. Review backward-compatible migrations. Prevent overlapping releases. Add post-swap production verification and a separate data-recovery plan.

Your current pipeline does not include automated database migration, production post-swap verification, automated rollback, full SQL integration tests, or a universal concurrency lock. Treat those as deliberate next improvements rather than assuming the pipeline already provides them.

## Hands-on exercises

1. Use trigger: none, run manually, and identify the source branch and commit.

2. Open each build task log and locate the actual dotnet operation.

3. Download drop and inspect the ZIP root structure.

4. Fail a test on a learning branch and observe later stages stop or skip.

5. Run a feature branch and explain why the deployment branch condition is false.

6. Configure a production environment approval and approve one controlled release.

7. Move nonsecret values into a variable group; authorize only this pipeline.

8. Verify Azure SQL connectivity and /health/ready before rerunning a release.

9. Add SQL-backed integration tests using isolated test data.

10. Introduce reviewed migrations, production verification, and release concurrency controls.

## Questions to ask for every pipeline line

What executes it? Where does it execute? What input does it require? What output does it produce? What should happen if it fails? Answering these five questions is the foundation of independent pipeline debugging.

# 14 / Official references and quick navigation

Use these Microsoft references when changing task inputs or investigating behavior. Task versions and portal labels can evolve; the repository remains the source of truth for your active configuration.

**Pipeline concepts**
https://learn.microsoft.com/en-us/azure/devops/pipelines/get-started/key-pipelines-concepts

**Build and deploy .NET**
https://learn.microsoft.com/en-us/azure/devops/pipelines/ecosystems/dotnet-core

**Trigger schema**
https://learn.microsoft.com/en-us/azure/devops/pipelines/yaml-schema/trigger

**UseDotNet task**
https://learn.microsoft.com/en-us/azure/devops/pipelines/tasks/reference/use-dotnet-v2

**.NET CLI task**
https://learn.microsoft.com/en-us/azure/devops/pipelines/tasks/reference/dotnet-core-cli-v2

**ArchiveFiles task**
https://learn.microsoft.com/en-us/azure/devops/pipelines/tasks/reference/archive-files-v2

**Pipeline artifacts task**
https://learn.microsoft.com/en-us/azure/devops/pipelines/tasks/reference/publish-pipeline-artifact-v1

**AzureWebApp task**
https://learn.microsoft.com/en-us/azure/devops/pipelines/tasks/reference/azure-web-app-v1

**Azure CLI task**
https://learn.microsoft.com/en-us/azure/devops/pipelines/tasks/reference/azure-cli-v2

**Slot management task**
https://learn.microsoft.com/en-us/azure/devops/pipelines/tasks/reference/azure-app-service-manage-v0

**Variable groups**
https://learn.microsoft.com/en-us/azure/devops/pipelines/library/variable-groups

**Service connections**
https://learn.microsoft.com/en-us/azure/devops/pipelines/library/connect-to-azure

**Approvals and checks**
https://learn.microsoft.com/en-us/azure/devops/pipelines/process/approvals

**App Service slots**
https://learn.microsoft.com/en-us/azure/app-service/deploy-staging-slots

## Portal shortcuts

Pipeline logs: Pipelines > pipeline > run > stage > job > task.
Variables: Pipelines > Library.
Connections: Project settings > Service connections.
Approvals: Pipelines > Environments > environment > Approvals and checks.
Slots: Azure Portal > App Service > Deployment slots.
Runtime settings: slot > Environment variables.
Application exceptions: slot > Log stream.